

Learning Optimal Blackjack Policies Using Reinforcement Learning

Kobi Uzu

Abstract

Blackjack provides a simple environment that has a relatively small state space with stochastic rewards that are well suited reinforcement learning algorithms. In this work, we investigate and compare the use of tabular reinforcement learning methods to learn near-optimal blackjack strategies under realistic casino rules. We implement a high-speed simulator written in C++ and compare the algorithms: Monte Carlo, SARSA, Expected SARSA, and Q-learning to the theoretical perfect strategy and random strategies. Our results show that Q-Learning and Expected SARSA were the most effective in learning and even exceeded the theoretical optimal structure.

1 Introduction

Blackjack is a stochastic decision-making problem in which players must balance risk and reward in an uncertain world. Although state transitions and rewards are stochastic, the underlying probability distributions are fixed, implying the existence of an optimal policy that maximizes expected return. This structure makes blackjack a natural testbed for reinforcement learning (RL), as learned policies can be directly compared against a known theoretical optimum.

The goal of this project is to evaluate how effectively certain tabular RL methods can learn optimal blackjack strategies and to analyze the impact of the rate of exploration and algorithm choice on learning performance.

2 Problem Formulation

We model blackjack as a fully observable, episodic, finite Markov Decision Process (MDP). Each episode corresponds to a single game against the dealer, with rewards assigned at the end of the episode as +1.5 for a natural blackjack win, +1 for a normal win, 0 for a draw, and -1 for a loss. Intermediate rewards are zero.

The state space consists of the player's hand total, dealer upcard, and other binary variables indicating the presence of a usable ace and the availability of doubling or splitting actions. The action space includes hit, stand, double, and split. We set the discount factor to $\gamma = 1$ due to the episodic nature of the game.

Unlike the original blackjack example in Sutton & Barto [1], which ignores doubling and splitting, our formulation includes these options. We assume continuous shuffling (effectively an infinite deck), allow doubling after splitting, require the dealer to hit on soft 17, and allow only one card to be drawn after splitting aces. This setup more closely matches standard casino rules and increases the complexity of the state space and decision-making process.

3 Methods

3.1 Reinforcement Learning Algorithms

We implemented four tabular RL algorithms: Q-learning, SARSA, Expected SARSA, and first-visit Monte Carlo. For SARSA, Expected SARSA, and Monte Carlo: actions were selected using a η -greedy policy, where the exploration rate for a state-action pair (s, a) was defined as

$$\epsilon(s, a) = \frac{\eta}{\eta + N(s, a)},$$

with $N(s, a)$ being the number of visits to (s, a) . This ensures that exploration decreases rationally as the agent gains experience, allowing the algorithm to converge. We also experimented with different values of η to observe how the level of exploration affects the learning performance.

The learning rate was set adaptively as

$$\alpha = \frac{1}{N(s, a) + 1}.$$

For Expected SARSA, the expected value of the next state under the η -greedy policy was computed as

$$\mathbb{E}[Q(s', a')] = (1 - \epsilon(s', a')) \max_{a'} Q(s', a') + \epsilon(s', a') \frac{1}{|\mathcal{A}|} \sum_{a'} Q(s', a'),$$

where $\epsilon(s', a') = \frac{\eta}{\eta + N(s', a')}$ is the exploration rate for the next state-action pair, $|\mathcal{A}|$ is the number of available actions, and the second term represents the mean Q-value over all actions to account for exploration. This allows Expected SARSA to correctly estimate the expected return given the probability of exploratory moves.

3.2 Handling Split Hands

The `split` action produces two independent hands. Rewards are determined only after both hands are completed.

Monte Carlo. Each split hand is treated as its own episode. First-visit Monte Carlo returns are computed recursively for each hand, and the total return for the original state-action pair is the sum of the returns from both hands. This ensures that the expected return is accurately reflected in the first-visit estimates.

Q-learning, SARSA, and Expected SARSA. For these algorithms, the update targets are modified to account for both resulting hands, s'_1 and s'_2 and where a'_1 and a'_2 are the next actions selected according to the policy in states s'_1 and s'_2 , respectively.

Q-learning:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left(r + \max_{a'} Q(s'_1, a') + \max_{a'} Q(s'_2, a') - Q(s, a) \right)$$

SARSA:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left(r + Q(s'_1, a'_1) + Q(s'_2, a'_2) - Q(s, a) \right)$$

Expected SARSA:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left(r + \mathbb{E}_{a'}[Q(s'_1, a')] + \mathbb{E}_{a'}[Q(s'_2, a')] - Q(s, a) \right)$$

where $\mathbb{E}_{a'}[Q(s', a')]$ is computed under the η -greedy policy, accounting for the probability of exploratory moves.

These modifications preserve the Markov property and ensure that the estimated return for a split action correctly reflects the contributions of both resulting hands, whether using off-policy, on-policy, or expected updates.

All RL algorithms were implemented in Python (and/or C++) and the code is publicly available at GitHub.

4 Experiments and Results

4.1 Experimental Setup

All four algorithms (Q-learning, SARSA, Expected SARSA, and first-visit Monte Carlo) were trained for 1,000,000 episodes using the adaptive learning rate $\alpha = 1/(N(s, a) + 1)$ and η -greedy exploration, where the exploration rate was defined as $\epsilon(s, a) = \eta/(\eta + N(s, a))$. For evaluation, we measured the average return per episode over 1,000,000 test games without exploration.

Performance of Different Reinforcement Learning Algorithms in Blackjack

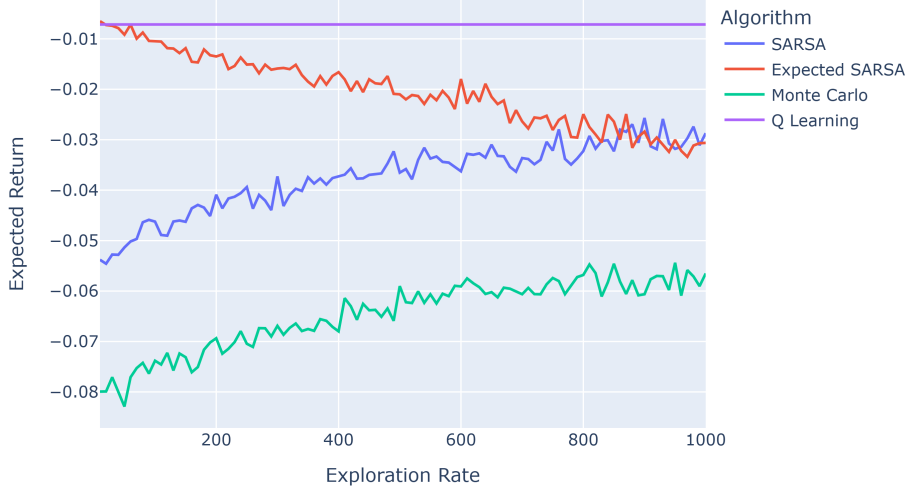


Figure 1: Plot of average return over 1,000,000 test games against exploration rate (η)

4.2 Results

Figure 1 shows the expected return as a function of the exploration rate η for all four algorithms. Q-learning produces a near-flat line, indicating that it converges quickly to the optimal strategy regardless of exploration, since it is an off-policy algorithm and always updates toward the maximal action value.

Expected SARSA exhibits a decline in performance as η increases. This likely occurs because Expected SARSA computes the expected return under the η -greedy policy, which already accounts for exploration in the update. Even with low exploration, it is able to propagate values effectively and converge to a near-optimal strategy. Increasing η unnecessarily weights suboptimal actions more heavily in the expectation, reducing average returns.

On the other hand, on-policy SARSA and first-visit Monte Carlo rely on the actions actually taken to update Q-values or returns. With very low exploration, the agent may fail to experience enough diverse state-action pairs, slowing convergence and preventing it from discovering optimal strategies. Increasing η improves performance by encouraging broader exploration, but only up to a point — once the agent explores too much, suboptimal actions dominate updates and performance plateaus or slightly declines. Overall, Expected SARSA outperforms SARSA across nearly all values of η . This is likely due to Expected SARSA’s inherent robustness to high-variance reward structures, such as those found in Blackjack. While standard SARSA relies on a single sampled action for its updates, Expected SARSA utilizes the expectation over all possible actions, smoothing out the noise in the environment.

Furthermore, Monte Carlo (MC) methods perform significantly worse than the Temporal Difference (TD) learners (SARSA, Expected SARSA, and Q-Learning). This performance gap is primarily because Monte Carlo methods require a completed episode to perform an update. In contrast, TD methods leverage bootstrapping, allowing them to learn from every state transition. This enables faster convergence and more efficient credit assignment compared to the delayed updates of Monte Carlo.

4.3 Long-run Evaluation

To obtain precise estimates of the true performance of the learned strategies, Q-learning and Expected SARSA were trained for 200 million episodes at a fixed exploration rate $\eta = 1$ and then tested over an additional 200 million episodes without exploration. This extremely large number of episodes ensures a very

narrow confidence interval on the measured expected returns.

The resulting strategies were compared to a reference basic strategy table obtained online. Both Q-learning and Expected SARSA slightly outperformed this reference strategy. This improvement is expected because the reference table was likely computed under slightly different casino rules (e.g., no doubling after split, a finite six-deck shoe, or dealer standing on soft 17), whereas our environment assumes continuous shuffling (infinite deck), doubling after split, and dealer hits on soft 17. By training directly in the rules we used, the RL algorithms learned strategies that better exploit these specific conditions.

Strategy	Description	Expected Return
Q-learning	Learned policy after 200M episodes	-0.518%
Expected SARSA	Learned policy after 200M episodes	-0.519%
Basic Strategy	Reference table from online source	-0.680%

Table 1: Comparison of the expected returns for Q-learning, Expected SARSA, and a reference basic strategy. Both RL algorithms slightly outperform the reference due to training under the specific rules of our environment.

5 Conclusion

This work demonstrates that tabular reinforcement learning methods can efficiently learn optimal blackjack strategies and that Expected SARSA and Q-Learning are the best algorithms for this specific problem. Future work includes extending the approach to function approximation, incorporating card counting, and analysing risk-sensitive reward formulations.

References

- [1] R. Sutton and A. Barto, *Reinforcement Learning: An Introduction*, MIT Press, 2018.